Newton School of Technology

12

# Lecture 12

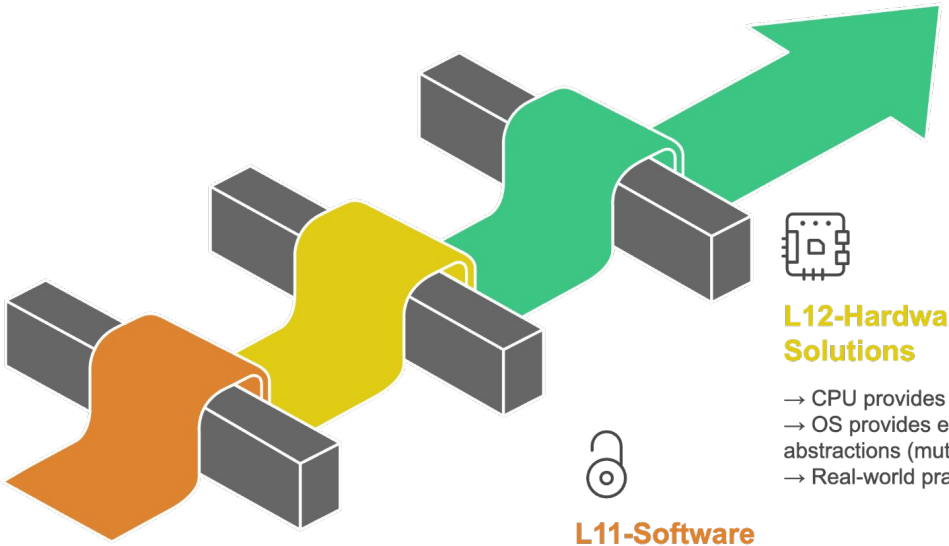
**Process Synchronization III: Hardware Support and Mutex Locks**

by - Brijesh Shukla

C07: Operating Systems

# THE STORY SO FAR

Newton School of Technology



**L12-Hardware Solutions**

→ CPU provides atomic operations
→ OS provides elegant abstractions (mutex)
→ Real-world practical solutions

**L11-Software Solutions**

Complex protocols, busy waiting, hard to reason about

**L10-The critical section problem**

Race conditions everywhere!
Chaos!

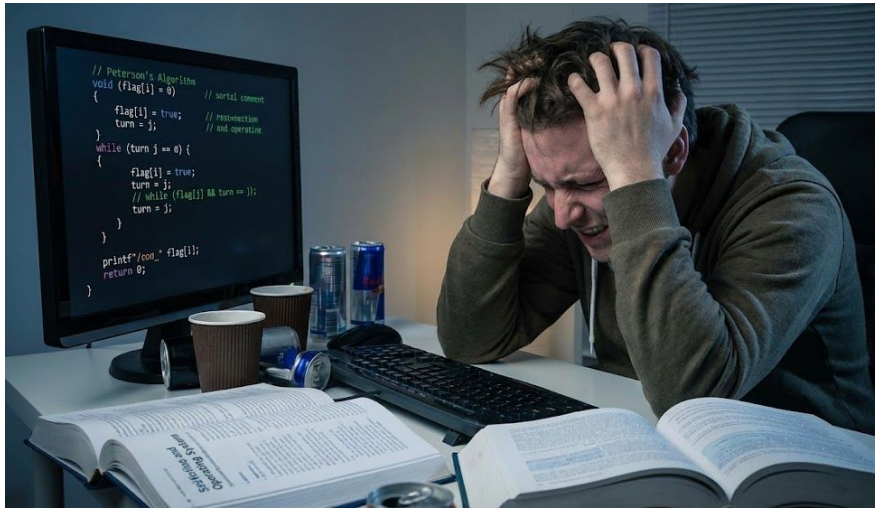
**The Question: Can the hardware make synchronization easier?**

# THE SOFTWARE SOLUTION PROBLEM

Newton School of Technology

**Problems with Peterson's and Dekker's:**

❌ Busy Waiting - CPU cycles wasted spinning
❌ Complex Logic - Hard to understand and debug
❌ Not Scalable - Try extending to 10 threads!
❌ No Atomicity Guarantee - Memory operations can be reordered
❌ Platform Dependent - May not work on all architectures




**Real-World Impact:**

- Your laptop battery draining while threads spin
- Wasted compute cycles on cloud servers ($$$)
- Race conditions on modern multi-core systems

**We need ATOMIC operations guaranteed by hardware!**

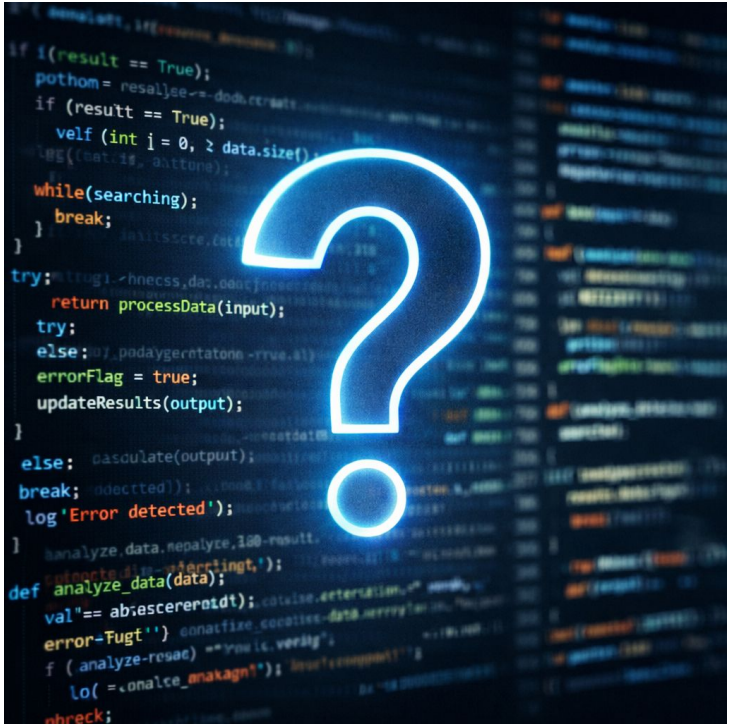
# Understanding the Limitation

Newton School of Technology

**Why can't we rely solely on software solutions like Peterson's algorithm in real-world systems?**

A) They're too fast
B) They waste CPU with busy waiting and don't scale
C) They're too simple
D) Hardware doesn't support them

**ANSWER: B) They waste CPU with busy waiting and don't scale**
Software solutions busy-wait (wasting cycles), are complex to implement correctly, don't scale beyond 2-3 threads, and lack guaranteed atomicity on modern hardware.



# ENTER THE HARDWARE

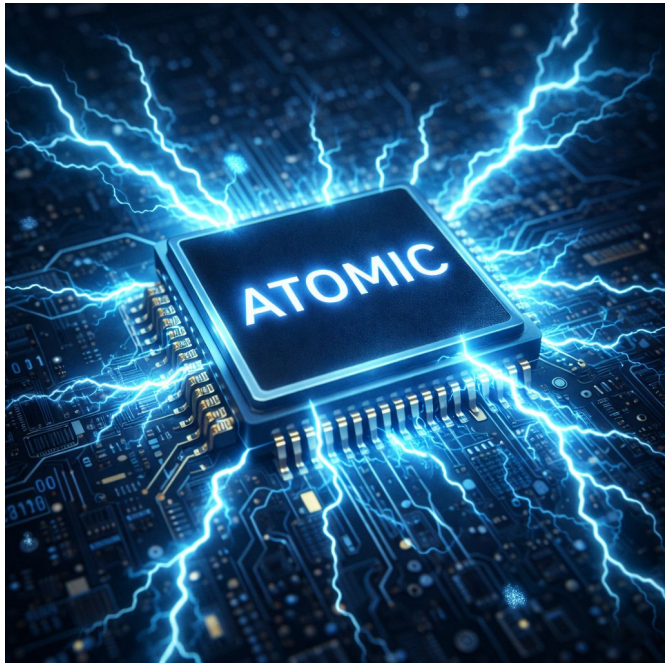
Newton School of Technology

**What if the CPU could do this in ONE uninterruptible step?**

Instead of:
1. Read lock value
2. Check if it's 0
3. Set it to 1
[Other thread can interrupt here! Race condition!]

Hardware Provides:
ReadAndSet in ONE atomic instruction!
→ Uninterruptible by other cores
→ Guaranteed by the CPU
→ Hardware magic ✨

**The Revolution: Test-and-Set (TAS) and Compare-and-Swap (CAS)**

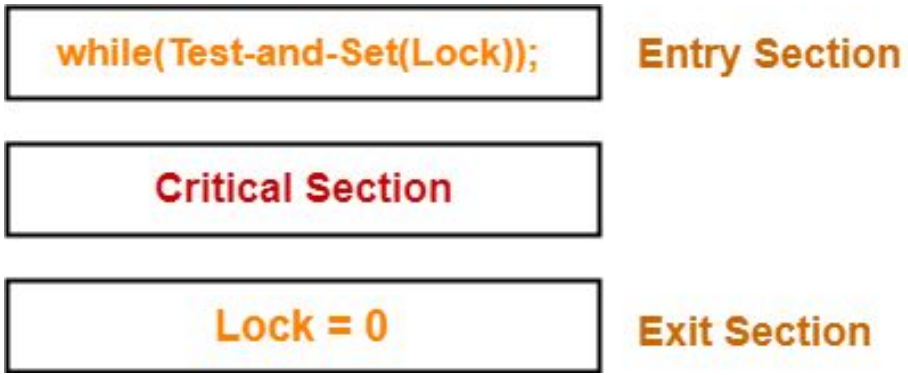


# TEST-AND-SET (TAS)

Newton School of Technology

**Test-and-Set (TAS):**

```
boolean TestAndSet(boolean *lock) {
   boolean old = *lock;  // Read current value
   *lock = true;         // Set to true
   return old;           // Return old value
}
// ENTIRE operation is ATOMIC (one CPU instruction!)
```




**How It Works:**
1. Returns the OLD value of the lock
2. Sets the lock to TRUE (locked state)
3. All in ONE uninterruptible instruction

**CPU Instructions:**
- x86: XCHG instruction
- ARM64 (Apple Silicon): LDAXR/STXR pair

# TAS IN ACTION – THE SPINLOCK

Newton School of Technology

**Implementing a Spinlock:**

```
// Shared lock variable
boolean lock = false;

// Acquire lock
void acquire() {
   while (TestAndSet(&lock))
      ; // Keep spinning if lock was already true
}

// Release lock
void release() {
   lock = false;
}
```


**The Magic:**

✓ Only ONE thread sees false and sets it to true
✓ All others see true and keep spinning
✓ Mutual exclusion guaranteed by hardware!

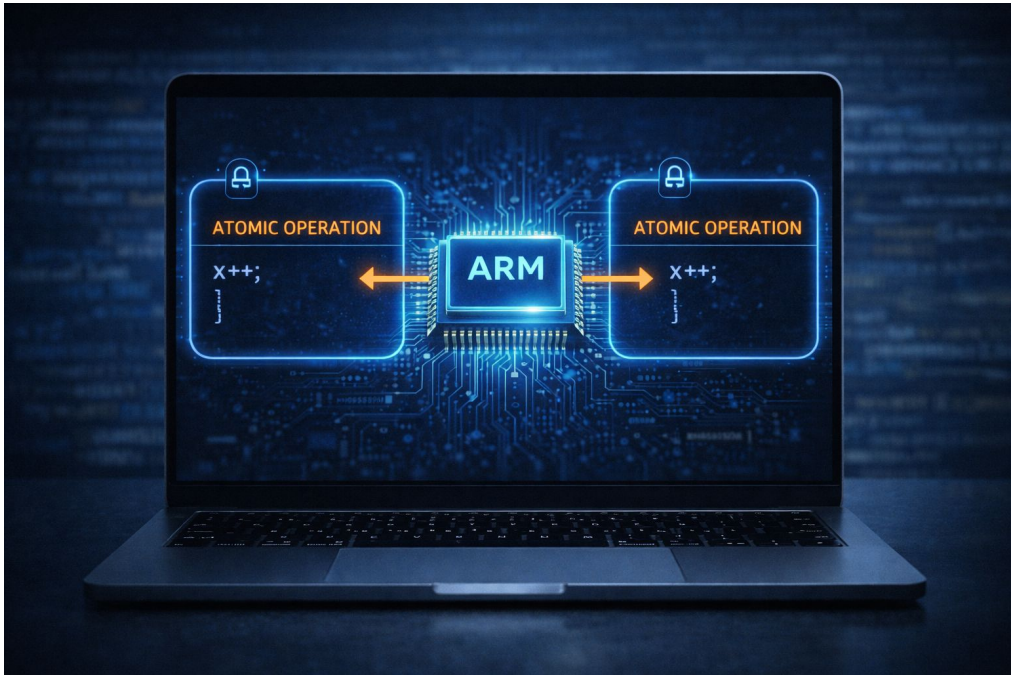
# TAS REAL-WORLD EXAMPLE

Newton School of Technology

**On macOS (Apple Silicon - ARM64):**

**Modern Approach - C11 Atomics:**

```
#include <stdatomic.h>
atomic_flag lock = ATOMIC_FLAG_INIT;
void acquire() {
  while (atomic_flag_test_and_set(&lock)); // spin
}
void release() {
  atomic_flag_clear(&lock);
}
```




**Under the Hood (ARM64):**

- LDAXR - Load Exclusive Register (with Acquire)
- STXR - Store Exclusive Register
- Hardware guarantees atomicity

# TAS ADVANTAGES & DISADVANTAGES

Newton School of Technology

**Advantages:**
✅ Simple to understand and implement
✅ Guarantees mutual exclusion (hardware-backed)
✅ No complex protocols like Peterson's
✅ Works correctly on multi-core systems

**Disadvantages:**
❌ Still uses busy waiting (wastes CPU!)
❌ No bounded waiting (starvation possible)
❌ Cache coherence traffic on multi-core
❌ Battery drain on mobile devices

**The Insight:** TAS solves correctness, but not efficiency!

**We still have the busy-waiting problem...**

# TAS Lock Simulation

Newton School of Technology

**Scenario:**
- lock = false initially
- T1 calls TAS at t=0
- T2 calls TAS at t=1
- T3 calls TAS at t=2
- T1 releases at t=10

**Tasks:**
1. Which thread acquires the lock first?
2. What do T2 and T3 do while waiting?
3. How many times do they spin (approximately)?
4. What happens when T1 releases at t=10?

**Draw a timeline showing TAS calls and results!**

**Solution:**

t=0: T1 calls TAS(lock)
→ lock was false, TAS returns false
→ lock set to true, T1 ENTERS critical section ✓

t=1: T2 calls TAS(lock)
→ lock is true, TAS returns true
→ T2 keeps spinning ⟳

t=2: T3 calls TAS(lock)
→ lock is true, TAS returns true
→ T3 keeps spinning ⟳

t=0-10: T2 and T3 spin continuously (busy waiting!)
→ Approximately thousands of spin iterations

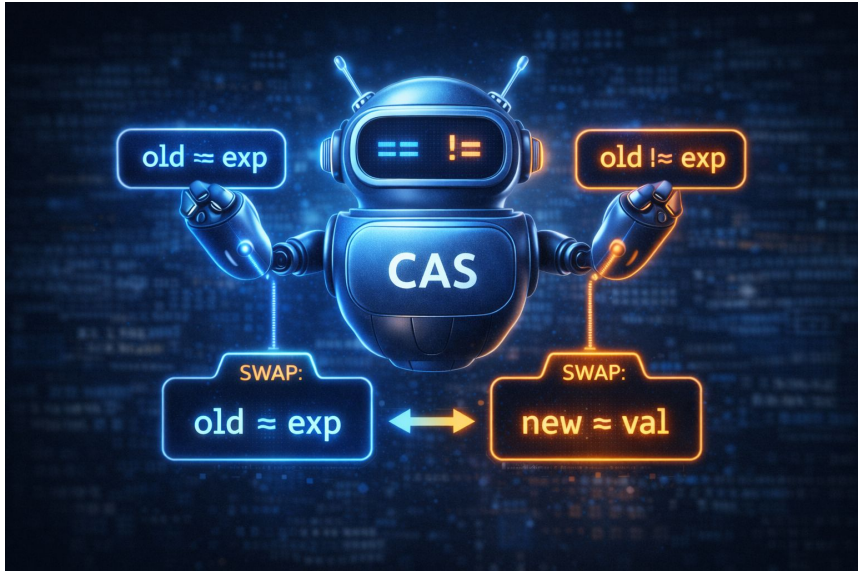
t=10: T1 calls release(), lock = false
→ Next TAS by T2 or T3 succeeds (whoever calls first)

# COMPARE-AND-SWAP (CAS)

Newton School of Technology

**Compare-and-Swap (CAS) - More Flexible!**

```
boolean CompareAndSwap(int *ptr, int expected, int
new) {
  int actual = *ptr;
  if (actual == expected) {
     *ptr = new;
     return true;  // Swap succeeded!
  }
  return false;     // Swap failed, someone changed it
}
// ENTIRE operation is ATOMIC
```




**Key Difference from TAS:**
- TAS: Always sets to true, returns old value
- CAS: Only sets if value matches expected, returns success/fail

**Power: Can check for SPECIFIC values, not just set blindly!**

# CAS ON DIFFERENT ARCHITECTURES

Newton School of Technology

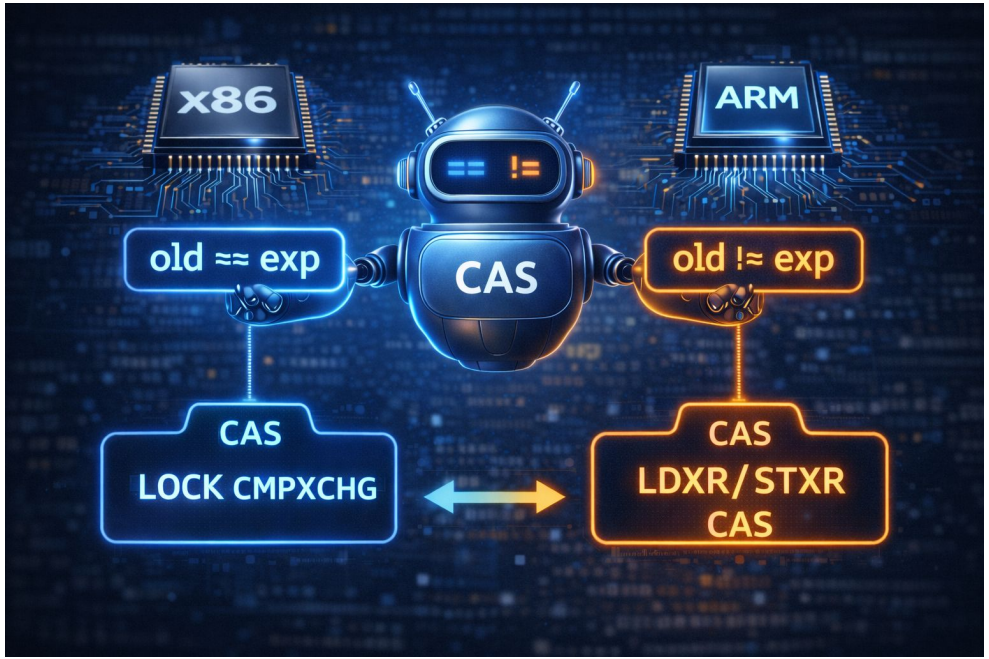
**CPU Instructions:**

**x86 (Intel/AMD):**
• CMPXCHG instruction
• Single atomic instruction

**ARM64 (Apple Silicon):**
• LDXR/STXR pair (Load-Exclusive/Store-Exclusive)
• Loop until exclusive access succeeds

**macOS/C11 Standard:**
• OSAtomicCompareAndSwapInt() (legacy)
• atomic_compare_exchange() (modern C11)

**All provide the same guarantee: Atomicity!**



# CAS FOR OPTIMISTIC LOCKING

Newton School of Technology

**Optimistic Concurrency with CAS:**

**Philosophy:**
- Assume NO conflict will occur
- Try to update without locking
- If someone else changed it, retry

**Traditional Lock (Pessimistic):**
1. Acquire lock (always block others)
2. Do work
3. Release lock

**CAS Approach (Optimistic):**
1. Read current value
2. Calculate new value
3. Try CAS - if it worked, done! If not, retry

**Best for: Low contention scenarios (conflicts rare)**

# CAS EXAMPLE – ATOMIC INCREMENT

Newton School of Technology

**Atomic Increment WITHOUT Traditional Locks:**

```
void atomic_increment(int *counter) {
   int old, new;
   do {
      old = *counter;          // Read current value
      new = old + 1;           // Calculate new value
   } while (!CompareAndSwap(counter, old, new));
   // Retry if another thread modified counter
}
```


**How It Works:**

1. Read counter (say it's 10)
2. Calculate new value (11)
3. CAS: "Is counter still 10? If yes, set to 11"
4. If another thread changed counter → CAS fails → retry
5. Eventually succeeds!

**No locks needed! This is lock-free programming!**

# THE ABA PROBLEM

Newton School of Technology

**The ABA Problem:**

Time 0: Thread 1 reads value A
Time 1: Thread 2 changes A → B
Time 2: Thread 2 changes B → A (back to original!)
Time 3: Thread 1's CAS succeeds! (sees A, expects A)
BUT Thread 1 doesn't know A changed and came back!

**The Problem:**
CAS only checks current value, not "has it changed?"

**Real-World Impact:**
• Lock-free data structures can corrupt
• Pointers reused can cause use-after-free

**Solutions:**
✓ Version numbers ($A_1$, $A_2$, $A_3$...)
✓ Tagged pointers (extra bits for version)
✓ Hazard pointers (mark in-use objects)

# TAS vs. CAS

Newton School of Technology

## TAS vs. CAS

| Aspect | Flexibility | Return Value | Use Case | Lock-Free Code | Complexity |
|---|---|---|---|---|---|
| TAS | Simple set | Old value | Basic locks | Difficult | Simpler |
| CAS | Conditional swap | Success/failure | Optimistic concurrency | Enables lock-free | More complex |

When to Use:
- TAS: Basic spinlocks, simple scenarios
- CAS: Lock-free data structures, optimistic updates, counters

**Both Provide: Hardware-guaranteed atomicity!**

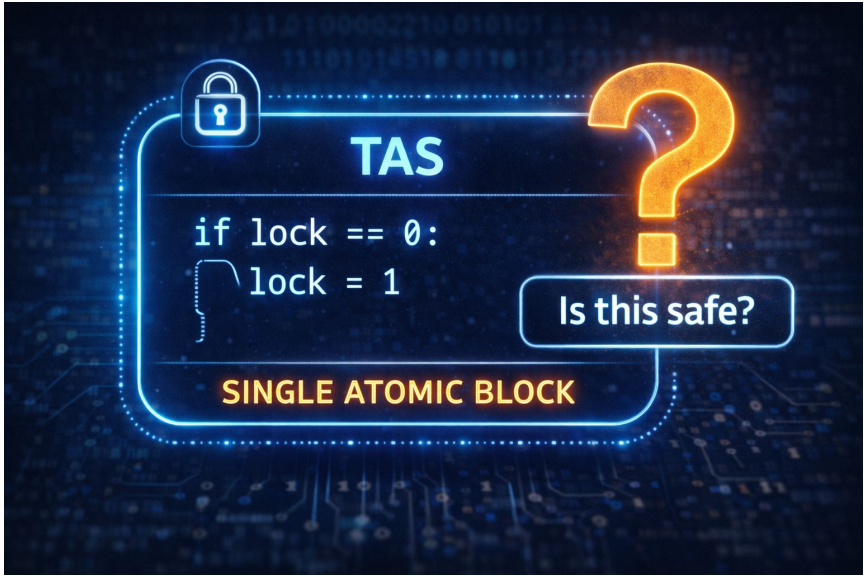
# TAS Understanding Check

Newton School of Technology

**What does Test-and-Set return, and what does it set?**

A) Returns true, sets to false
B) Returns the old value, sets to true
C) Returns the new value, sets to old value
D) Returns false, sets to the thread ID

**ANSWER: B) Returns the old value, sets to true**
TAS atomically reads the current value (returns it) and then sets the lock to true (locked state). This lets the caller know if the lock was free (returned false) or taken (returned true).



# THE EFFICIENCY PROBLEM REMAINS

Newton School of Technology

**We Still Have a Problem...**

**TAS and CAS Solved:**
✅ Correctness (mutual exclusion guaranteed)
✅ Simplicity (easier than Peterson's)
✅ Multi-core support (hardware atomicity)

**But We STILL Have:**
❌ Busy Waiting - CPUs spin in while loops
❌ Wasted Energy - Battery drain
❌ Wasted Cycles - Could run other threads!

**The Question:**
What if instead of spinning, threads could SLEEP and be WOKEN when the lock is free?

**Enter: The Mutex (Mutual Exclusion Lock)**

# INTRODUCING THE MUTEX

Newton School of Technology

**What is a Mutex?**

**Mutex = Mutual Exclusion Lock**
- OS-provided synchronization primitive
- Built on hardware primitives (TAS/CAS)
- Adds sleeping/waking mechanism
- Available through pthread library

**Key Difference:**

**Spinlock:**
Thread → TAS → If locked, SPIN (busy-wait) ⟳

**Mutex:**
Thread → Lock attempt → If locked, SLEEP 😴
Lock released → OS WAKES thread → Thread runs ✓

**Result: No wasted CPU cycles!**

# MUTEX BASIC OPERATIONS

Newton School of Technology

**Basic Mutex Operations (macOS/POSIX):**

```
#include <pthread.h>
pthread_mutex_t lock;
// 1. Initialize
pthread_mutex_init(&lock, NULL);
// 2. Acquire lock (blocks if unavailable)
pthread_mutex_lock(&lock);
// 3. Critical section
shared_data++;
// 4. Release lock
pthread_mutex_unlock(&lock);
// 5. Cleanup
pthread_mutex_destroy(&lock);
```

**Simple API, powerful guarantees!**

# HOW MUTEX WORKS

Newton School of Technology

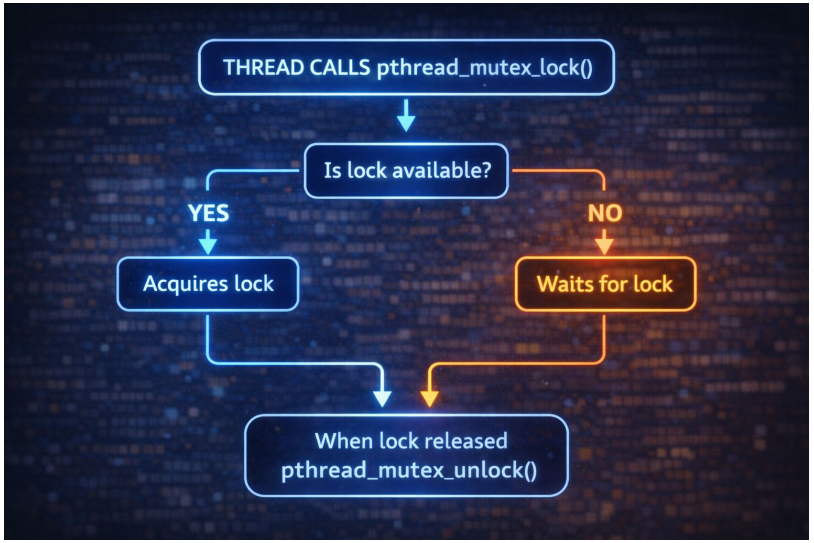
**Mutex Acquisition Process:**

**Thread calls pthread_mutex_lock():**

**Is lock available?**
**YES** → Acquire it, enter critical section ✓
**NO** → Add thread to waiting queue
→ Thread state: RUNNING → BLOCKED
→ OS schedules another thread
→ Thread SLEEPS (not wasting CPU) 😴

**When lock released (pthread_mutex_unlock()):**
→ OS checks waiting queue
→ Wakes ONE thread (removes from queue)
→ Awakened thread: BLOCKED → READY → RUNNING
→ Awakened thread acquires lock ✓

**Key: Thread gives up CPU when waiting!**



# CAS vs TAS Check

Newton School of Technology

**How is Compare-and-Swap different from Test-and-Set?**

A) CAS is faster
B) CAS checks for a specific expected value before swapping
C) CAS doesn't require hardware support
D) CAS can work with multiple threads, TAS cannot

**ANSWER: B) CAS checks for a specific expected value before swapping**
CAS takes an expected value and only performs the swap if the current value matches the expected value. TAS simply always sets to true and returns the old value. This makes CAS more flexible for optimistic concurrency patterns.

# LIVE DEMO – MUTEX IN ACTION

Newton School of Technology

Demo: https://my.newtonschool.co/playground/code/td06oli0xdk9

# MUTEX ADVANTAGES

Newton School of Technology

**Advantages of Mutex over Spinlock:**

✅ No CPU Wasted - Threads sleep, don't spin
✅ Better for Long Critical Sections - Minutes, hours OK!
✅ Scalable to Many Threads - OS manages queues efficiently
✅ Fairness - Waiting queue prevents starvation
✅ Priority Inheritance - Prevents priority inversion
✅ Works with I/O - Can block during I/O operations

**When to Use Mutex:**

- User-space applications (99% of the time!)
- Critical sections with significant duration
- I/O operations inside critical section
- Many threads competing for lock

**Default Choice:** Use mutex unless you have a specific reason not to!

# Mutex Understanding Check

Newton School of Technology

**What is the main advantage of a mutex over a spinlock?**

A) Mutexes are faster
B) Mutexes block/sleep instead of busy-waiting
C) Mutexes don't require hardware support
D) Mutexes prevent all deadlocks

**ANSWER: B) Mutexes block/sleep instead of busy-waiting**
When a mutex is unavailable, the requesting thread is put to sleep (blocked state) by the OS, yielding the CPU to other threads. Spinlocks busy-wait, wasting CPU cycles. This makes mutexes far more efficient for most scenarios.

# MUTEX IMPLEMENTATION DETAILS

Newton School of Technology

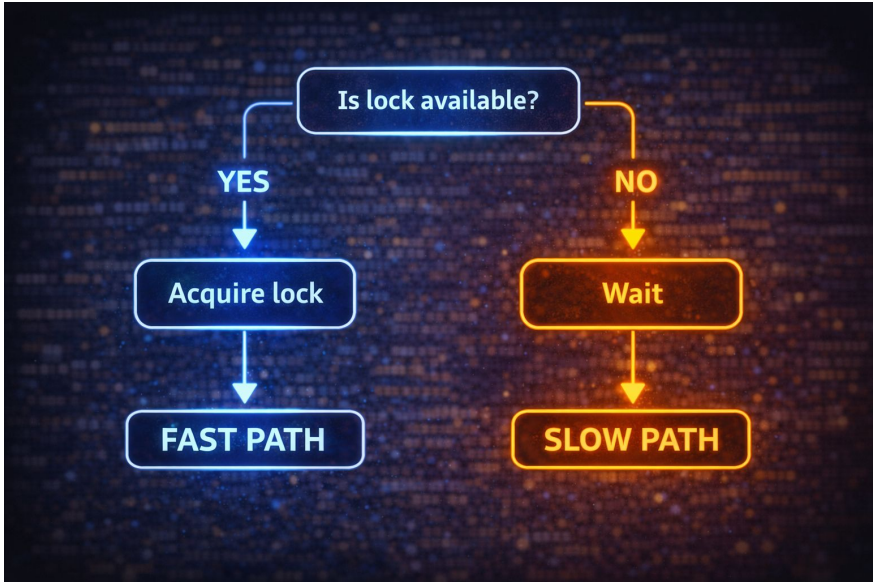
**Mutex Implementation: Hybrid Approach**

**Phase 1 - Fast Path (Userspace):**
- Try to acquire using atomic operation (CAS/TAS)
- If successful: No kernel involvement! ✓
- Cost: ~10 nanoseconds

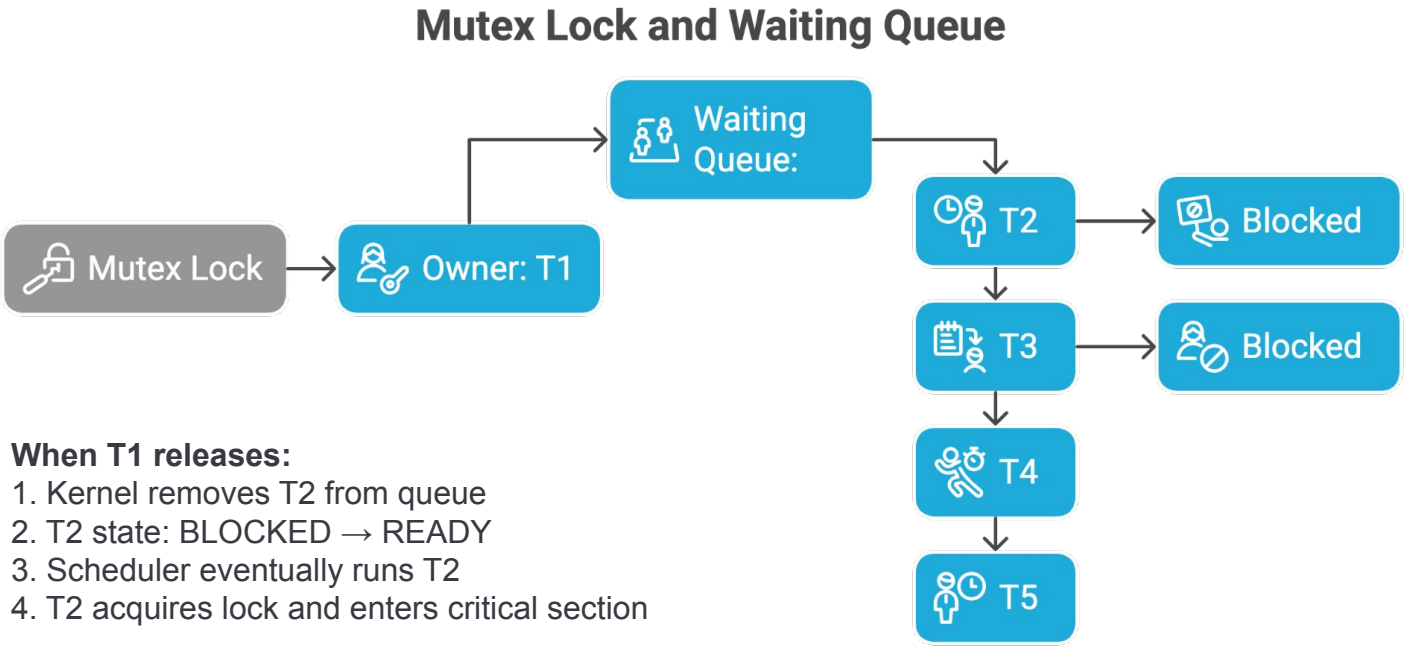
**Phase 2 - Slow Path (Kernel):**
- If lock unavailable: syscall to kernel
- Kernel adds thread to waiting queue
- Thread context-switched out (blocked state)
- Cost: ~1-2 microseconds (100x slower!)

**Optimization:**
Most locks are uncontended (fast path succeeds)
Only pay kernel cost when there's actual contention

**This is why mutexes can be efficient!**



# MUTEX WAITING QUEUE




**When T1 releases:**
1. Kernel removes T2 from queue
2. T2 state: BLOCKED → READY
3. Scheduler eventually runs T2
4. T2 acquires lock and enters critical section

**Fair:** First-in, first-out (FIFO) ordering
**No starvation:** Everyone eventually gets the lock

# PRIORITY INVERSION PROBLEM

Newton School of Technology

**The Priority Inversion Scenario:**

1. Low-priority Thread (T_low) acquires mutex
2. High-priority Thread (T_high) needs mutex, BLOCKS
3. Medium-priority Thread (T_med) preempts T_low
4. T_high waits while T_med runs!
   (High-priority waiting for low-priority indirectly!)

**Priority Order:** T_high > T_med > T_low
**Expected:** T_high runs first
**Reality:** T_high waits for T_low to finish, but T_med blocks T_low!

**Famous Example:**
Mars Pathfinder (1997) - rover kept resetting due to priority inversion!

**Solution: Priority Inheritance**
→ Temporarily boost T_low's priority to T_high's priority
→ T_low finishes quickly, releases lock
→ T_high can run

# MUTEX STATES

Newton School of Technology

**Three Mutex States:**

**State 1: UNLOCKED**
- No owner
- Available for acquisition
- Waiting queue empty

**State 2: LOCKED (Uncontended)**
- Owner thread exists
- NO threads in waiting queue
- Most common case (90%+ of the time!)

**State 3: LOCKED (Contended)**
- Owner thread exists
- One or more threads in waiting queue
- Expensive state (kernel involved)


**Transitions:**
UNLOCKED —(lock)→ LOCKED (Uncontended)
LOCKED (Uncontended) —(another lock attempt)→ LOCKED (Contended)
LOCKED —(unlock)→ UNLOCKED (if queue empty)
LOCKED (Contended) —(unlock)→ LOCKED (Uncontended) or UNLOCKED

# SPINLOCKS VS MUTEXES

Newton School of Technology

| Aspect | Spinlock | Mutex |
|---|---|---|
| Waiting | Busy-wait | Sleep/block |
| CPU Usage | Wastes cycles | Yields CPU |
| Overhead | Low (~10ns) | Higher (~1-2µs) |
| Best For | <1 microsecond | >1 microsecond |
| Contention | Poor | Good |
| Scalability | Poor | Excellent |
| Fairness | No guarantee | FIFO queue |
| Use Case | Kernel code | User applications |
| Battery Impact | High drain | Efficient |

**The Rule of Thumb:**

- Critical section < 1 microsecond → Consider spinlock
- Critical section > 1 microsecond → Use mutex
- Not sure? Use mutex!

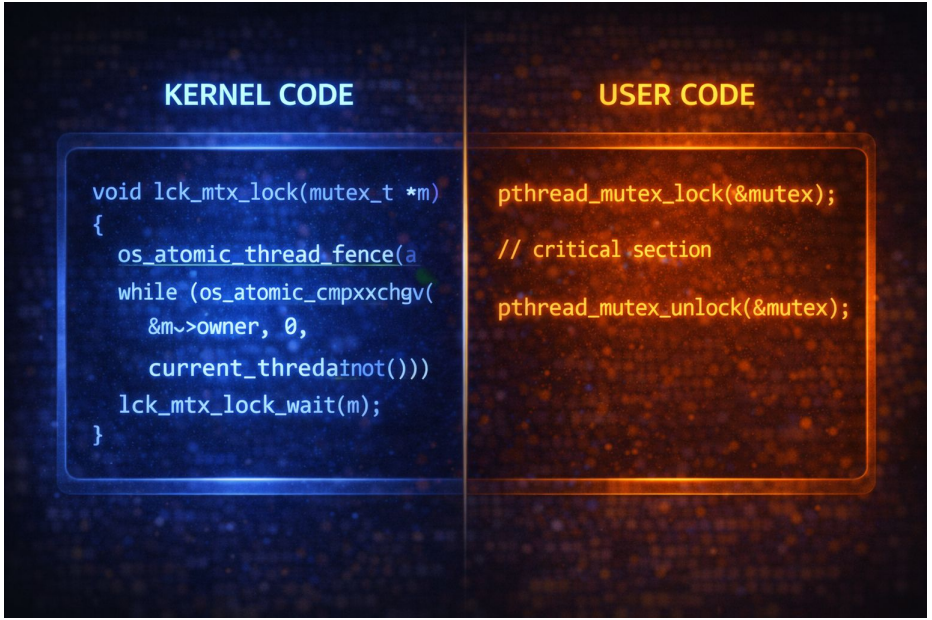
# When to Spinlock Check

Newton School of Technology

**When would you prefer a spinlock over a mutex?**

A) For long critical sections
B) For short, low-contention critical sections
C) When many threads compete
D) In user-space applications with I/O

**ANSWER: B) For short, low-contention critical sections (<μs)**
Spinlocks are appropriate when the critical section is very short (microseconds) and contention is low, as the cost of blocking (syscall + context switch) would exceed the spin time. They're commonly used in kernel code where sleeping isn't allowed.



# WHEN TO USE SPINLOCKS

Newton School of Technology

**Scenarios for Spinlocks:**

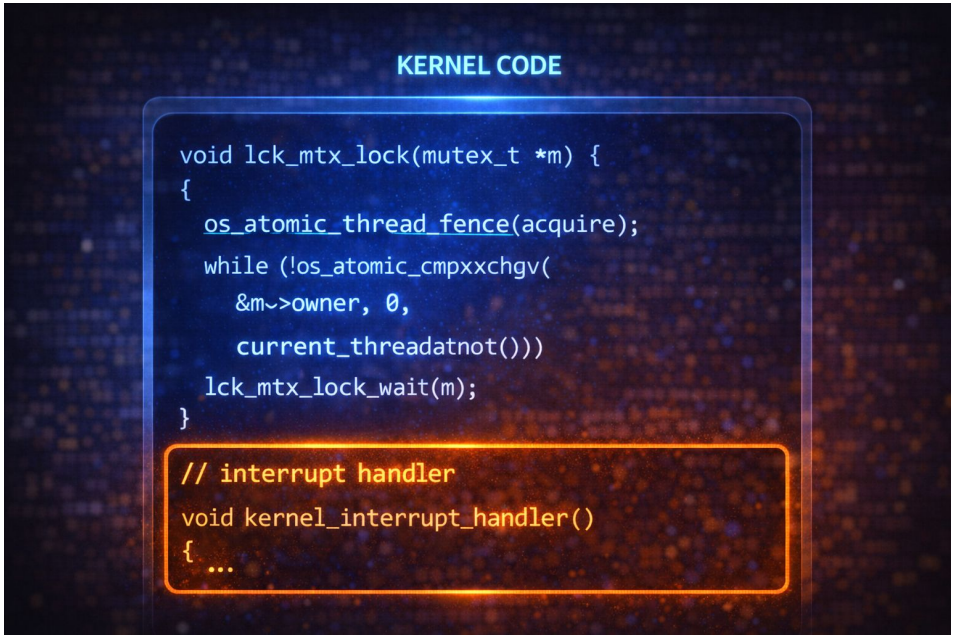
✓ **Critical Section is Very Short**
→ Few CPU instructions (nanoseconds)
→ Cost of blocking exceeds spin time

✓ **In Kernel Code**
→ Interrupt handlers can't sleep!
→ Some kernel paths where sleeping not allowed

✓ **Real-Time Systems**
→ Predictable timing requirements
→ No unpredictable wake-up delays

✓ **Low Contention**
→ Rarely conflict with other threads
→ Spin time minimal

✓ **Already Holding Other Locks**
→ To avoid deadlock from nested blocking



# WHEN TO USE MUTEXES

Newton School of Technology

**Scenarios for Mutexes:**

✓ **Critical Section Involves I/O**
→ File operations, network calls
→ Would waste CPU spinning during I/O

✓ **Significant Computation**
→ More than a few microseconds
→ Better to let other threads run

✓ **Many Threads Competing**
→ High contention scenarios
→ Waiting queue provides fairness

✓ **User-Space Applications**
→ Default choice for 99% of applications!
→ Battery-efficient on mobile

✓ **When Fairness Matters**
→ FIFO queue ensures no starvation


**Examples:** Web servers, databases, GUI applications, all normal programs!

# HYBRID APPROACHES

Newton School of Technology

**Adaptive Locks: Spin-Then-Block**

**Strategy:**
1. Spin in userspace for N iterations
2. If still locked, call blocking syscall
3. Sleep in kernel waiting queue

**Benefits:**
✓ Fast for short waits (avoid syscall)
✓ Efficient for long waits (don't waste CPU)
✓ Adapts to workload automatically

**Real-World Implementations:**

**macOS:** os_unfair_lock
• Replaces deprecated OSSpinLock
• Spins briefly, then blocks
• Prevents priority inversion

**Linux:** pthread_mutex with spin count
• Configurable spin iterations
• Falls back to kernel on timeout


**Modern Best Practice:** Use adaptive locks (os_unfair_lock on macOS)

# PERFORMANCE CONSIDERATIONS

Newton School of Technology

## Performance Analysis: Spinlock vs. Mutex

| Characteristic | Spinlock | Mutex |
|---|---|---|
| Acquire (Uncontended) | ~10 nanoseconds | ~10 nanoseconds (fast path) |
| Acquire (Contended) | N/A | ~1-2 microseconds (slow path) |
| Spin Iteration | ~5 nanoseconds per loop | N/A |
| Context Switch | N/A | ~1-10 microseconds |
| Syscall Overhead | No syscall overhead | Syscall overhead |
| CPU Usage (Waiting) | Wastes CPU | Other threads can run |
| Break-Even Point | Critical section < spin time (often <1µs) | Critical section > syscall cost (often >1µs) |

**Cache Effects:**

- Thread migration destroys cache locality
- Lock holder on different CPU → cold cache
- Contended locks → cache coherence traffic

# Beyond Basic Locks

Newton School of Technology

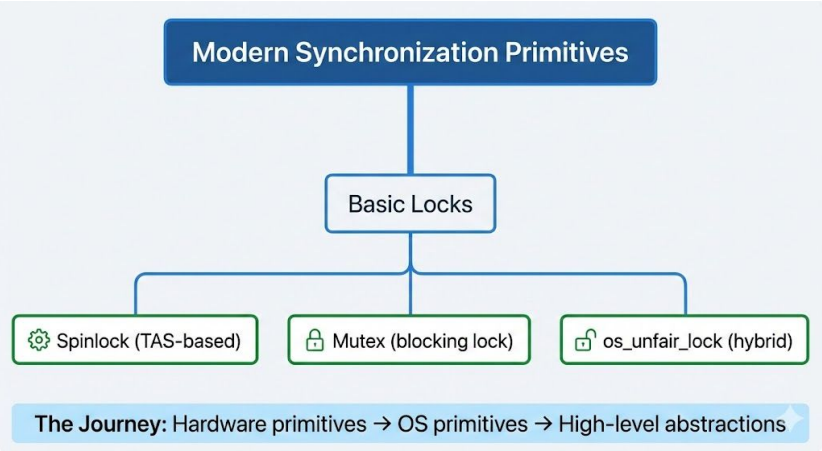
**Modern Synchronization Primitives:**

**Basic Locks:**
- Spinlock (TAS-based)
- Mutex (blocking lock)
- os_unfair_lock (hybrid)

**Advanced Primitives:**
- Read-Write Locks (multiple readers, single writer)
- Semaphores (counting synchronization) ← Next lecture!
- Condition Variables (wait for specific conditions)

**Lock-Free Techniques:**
- CAS-based data structures
- Wait-free algorithms
- Transactional memory (Intel TSX - now disabled)

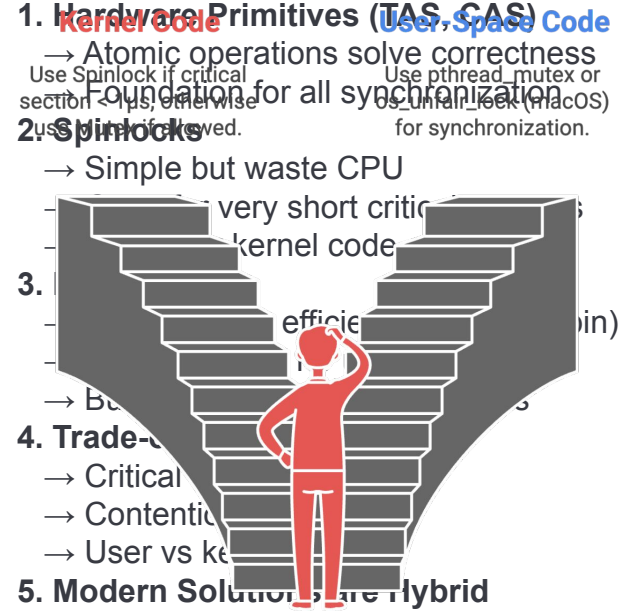
**High-Level Abstractions (macOS):**
- Grand Central Dispatch (GCD)
- dispatch_once (thread-safe singleton)
- dispatch_barrier (reader-writer coordination)



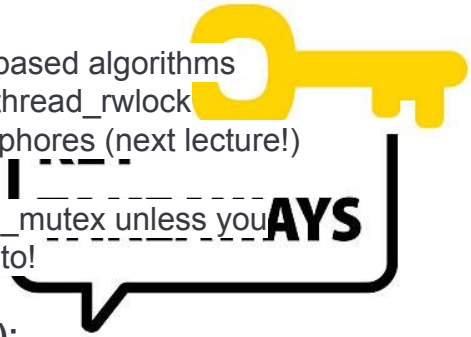
# Key Takeaways

## What Should YOU Use?

1. **Hardware Primitives (TAS, CAS)**
   → Atomic operations solve correctness
   → Foundation for all synchronization
2. **Spinlocks**
   → Simple but waste CPU
   → very short critic
   → kernel code
3. **[illegible]**
   → efficie in)
   → Bu
4. **Trade-**
   → Critical
   → Contentio
   → User vs ke
5. **Modern Solution[illegible]re Hybrid**
   → Adaptive locks (spin-then-block)
   → Best of both worlds




**Special Cases:**
- Need lock-free? → CAS-based algorithms
- Need reader-writer? → pthread_rwlock
- Need counting? → Semaphores (next lecture!)




**Default Rule:** Use pthread_mutex unless you have a specific reason not to!

**Red Flags (Don't do this!):**
- ❌ Spinlocks in user-space for long critical sections
- ❌ Mutexes in interrupt handlers
- ❌ Lock-free code without understanding ABA problem
- ❌ Rolling your own synchronization primitives

# Thank You!

Newton School of Technology

Thank you for your attention. Feel free to ask questions or revisit the topics covered.